Lab 1: Bluetooth Low Energy

General Instructions

Two important concepts of Bluetooth Low Energy (BLE) communication are the Generic Access Profile (GAP) and the Generic Attribute Profile (GATT). GAP controls the advertising, i.e. how the device is visible to the world, and the way devices interact with each, i.e. whether they connect to have bi-directional communication or exchange data via broadcasting without establishing a connection. The four roles defined in GAP are broadcaster, observer, peripheral, and central. In principle, BLE devices can operate in multiple GAP roles simultaneously if their hardware and BLE stack supports it. When a connection between two devices has been made, GATT manages the structured exchange of data through nested objects called characteristics, services, and profiles. Characteristics are identified by a pre-defined UUID and contain a single data point (which may also be an array of values). A service encapsulates one or more characteristics and has again a UUID for identification. A profile is a pre-defined collection of services. Have a look at the slides related to this lab for more details.

In this lab, we will implement two IoT devices that communicate via BLE. The first device collects data from sensors that are used for environmental monitoring (e.g. air quality, humidity, temperature, ...) and relays the data to a second device. You are welcome to implement this on real hardware, using real sensor data, but, for the purposes of this lab, you can also simulate fake data on the first device with random values in a reasonable range.

Your group can get a total of 10 points in this lab: 8 points for the implementation and evaluation, 2 points for the presentation. Remember that those groups that will not present this lab at the lab session have to upload a 6 to 8 min long video presentation with their submission in Moodle.

You can use the provided Renode simulation script (`.resc` file) as a basis, but you will have to modify at least the paths to the binaries of the IoT devices.

Be aware that PlatformIO does not support the newest version of Zephyr with the Nordic nRF52 platform. The currently supported version is 2.7.1. If something you got from the documentation of the latest version doesn't work, have a look at the older documentation.

Part 1 – Broadcaster and Observer (2 points)

We will start by using the broadcaster-observer pattern, where the device that collects the sensor data broadcasts the measurements so that surrounding observers can receive them. The two devices we are using are in range of each other and can, thus, directly communicate. The broadcasted data is part of the advertising data.

Your task is to set up the two devices using Zephyr and configure them with the appropriate GAP roles. Use at least one measured variable whose (fake) data is to be broadcasted by one device. The other device should receive the data, parse it, and print it to the serial monitor.

To verify your implementation, simulate this with Renode (using the provided `.resc` file) or deploy it on real hardware.

Hint 1: If you are not sure where to begin, have a look at the BLE examples of Zephyr (`https://github.com/zephyrproject-rtos/zephyr/tree/main/samples/bluetooth`).

Hint 2: Take a look at `bt_data_parse` *for parsing the data at the observer.*

Part 2 – Peripheral and Central (4 points)

Now we will implement two IoT devices that connect to each other and use GATT for structured communication. The devices are again in range of each other and can directly communicate. The device with the (fake) environmental data is the peripheral and the other one is the central.

Again, set up the two devices using Zephyr and the appropriate GAP roles. Establish a connection between peripheral and central. Implement the transfer of the same measured variable(s) as before using characteristics, a service, and a profile. You are welcome to use a suitable pre-defined service and pre-defined characteristics from the Bluetooth SIG (`https://www.bluetooth.com/specifications/assigned-numbers/`). If you want to, you can also implement your own service and characteristics.

Hint: The Zephyr BLE examples are again a good starting point.

Part 3 – Evaluation (2 points)

To evaluate the reliability and latency of your system, compare sent and received data packets and their timestamps. You can choose if you want to do this for broadcaster and observer or peripheral and central. It is enough to analyze this for one combination.

If you do this in Renode, uncomment line 9 of the provided `.resc` file to introduce probabilistic packet loss in relation to the distance between nodes. The simulation script also exposes two UART terminals `term1` and `term2` over TCP sockets with the ports `60001` and `60002`. You can use this to read data from them, e.g. with Python, to process it further and make your evaluation easier.

Create plots to show the reliability and latency of the data transfer between the two devices. Compare at least two different settings by, for example, changing the distance between the nodes in the simulation script.